

From a one-line brief to a dev-ready payments feature, with AI working as my product team

Digvijay Singh Nayal · Product Manager

Context: product assignment set by Zippee, a quick-commerce logistics company for D2C brands · July 2026

The area where I get the most out of AI is **turning an ambiguous problem into a build-ready package**: research, spec, design system, clickable prototype and a pre-development audit. I don't use AI to write faster. I use it as a team of specialists that I brief, challenge and audit, so my own time goes into the decisions that need judgment.

10+

rider and commerce apps benchmarked (Blinkit, Zepto, Swiggy, Flipkart, BigBasket and more)

v4.0

PRD after four pressure-tested versions; 12 assumptions and 9 decisions logged

22

coded UI components (React + TypeScript), synced into Claude Design

39

issues found by an AI audit of the prototype against the PRD

5

blockers caught **before** development, incl. a UPI double-collection race

THE BRIEF

A customer ordered 5 items on cash or pay-on-delivery. At the door they say: *"I only want 3."* Nothing in the rider app supports this. Design how the rider completes it, and make it **ready to move to development**.

WHY IT'S HARD

It looks like a UI feature, but it's actually a **payments and inventory problem**: the amount changes seconds before collection, over UPI on patchy 4G, for 150+ brands whose prices and coupons Zippee doesn't own, and every returned item has to make it back to the shelf.

What I delivered

ARTIFACT	WHAT'S IN IT
UI/UX research doc	Ground truth on Zippee's live rider app (Play Store listing, reviews, June 2026 revamp), rider persona, the full rider journey benchmarked and mapped screen by screen, 8 anti-patterns seen in the field
PRD v4.0 (ready for dev)	Doorstep flow S1 to S7, 4 state machines, payment logic with a worked example, 15 edge cases, internal API contracts, guardrails, NFRs, instrumentation, phased launch with kill criteria
Design system + component library	Tokens pixel-sampled from the live app, 22 components as code, interaction rules (56 px targets, Hindi-ready, no typeable amount field anywhere)
Clickable prototype	Rider flow S1 to S7 plus Home, Success and RTO screens; POD and prepaid variants; geofence scenarios; English with Hindi on the entry screen
AI-run UAT review	Prototype audited line by line against the PRD: 5 blockers, 7 high, 14 medium, 13 polish items, each with a fix and a gate

TRY THE PROTOTYPE

[Link added once the prototype is published](#)

Opens in any browser. Best viewed on a phone or in a narrow window. Demo data only.

01 The problem, and the insight that shaped everything

Zippee delivers for 150+ D2C brands from ~125 dark stores in 21 cities (public figures). COD dominates. Today every partial rejection turns into a phone call.

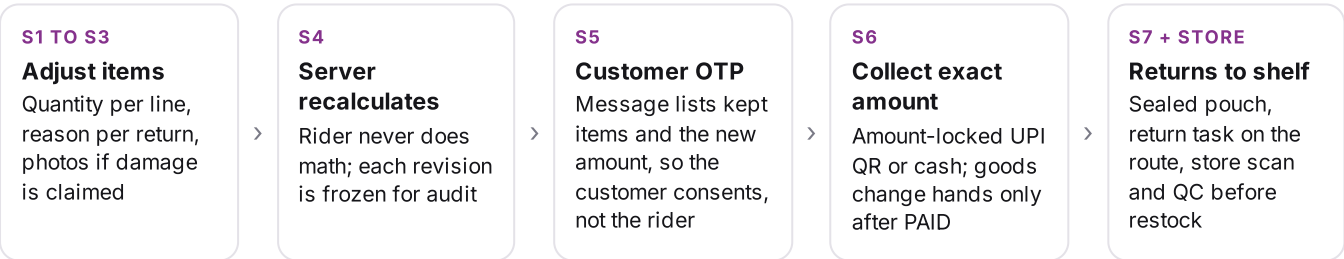
WHO	WHAT GOES WRONG TODAY
Zippee ops	Every partial is a 6 to 10 min ops call. Cash collected no longer matches system records, so daily reconciliation breaks. Store stock drifts because returns have no record.
Brand	Full RTO where a partial could have saved the sale. Website stock goes wrong, which leads to overselling and settlement disputes.
Customer	Waits while ops improvises, then pays a hand-calculated, unverified amount with no record of what went back.
Rider	Does mental math on money, gets a salary deduction when it's wrong, and runs late for the rest of the route.

Nobody renegotiates payment at the door. Blinkit, Zepto and BigBasket route around it with prepaid wallets. Zippee has no wallet (the brand owns the customer), so the only clean path is to collect the exactly-right amount the first time.

This insight came out of the AI-assisted benchmark (section 02). It changed the design brief from "build a returns screen" to "build a trustworthy re-pricing and collection system with a returns tail".

The solution in one line

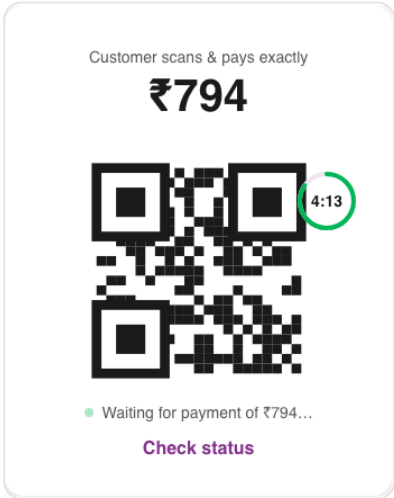
The rider marks what's being returned, the **server** recalculates the bill, the **customer** approves by OTP, the rider collects the **exact** amount, and every returned item is tracked back to the store.



The money math (worked example from the PRD)

ORIGINAL	A ₹500×2 B ₹300×1 C ₹100×2 coupon -₹150 · delivery ₹49 · COD ₹25 to pay ₹1,424
RETURNED	1×A and 2×C
KEPT	A: ₹500 - ₹50 coupon share = ₹450 B: ₹300 - ₹30 coupon share = ₹270 fees unchanged: ₹49 + ₹25 to pay ₹794

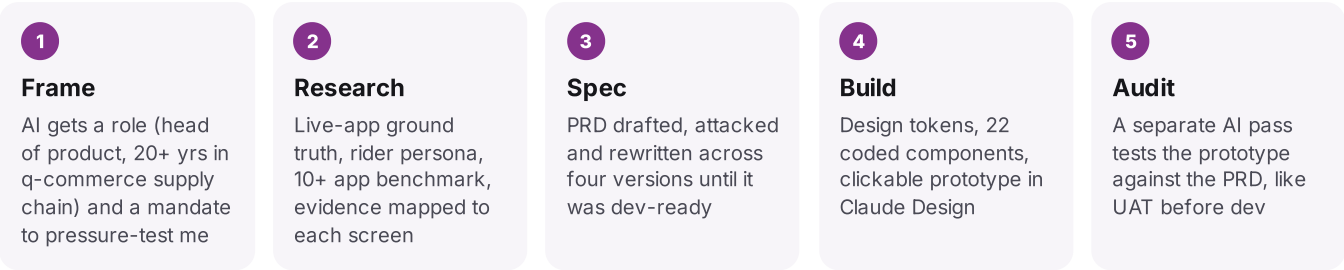
Coupon split uses the platform's own line-level allocation when it's sent (e.g. Shopify), else prorated by line value, rounded to the paisa with the remainder on the highest-value line, so totals always reconcile.



Amount-locked UPI QR with a 5-minute expiry ring, from the coded component library.

02 How I used AI: a five-stage pipeline

Every artifact is the input contract for the next one. AI does the heavy lifting inside each stage, and I own the handoffs between stages.



Audit findings loop back into the spec and the build. Gates: blockers fixed before dev pickup, high-severity items fixed before rider usability testing.

STAGE	WHAT AI DID	WHAT I DECIDED
Research	Read Zippee's Play Store listing and reviews, found the com.zippee.ridercod package ("COD" is in the name), collected field studies on Blinkit and Zepto riders, and pulled patterns from Flipkart Open Box, BigBasket and Uber India	Which lessons transfer and which don't. Zippee ≠ Blinkit: the brand owns the customer, COD dominates, and handover has minutes, not seconds. That gap is the strategic room this feature exploits.
Spec	Drafted the PRD, generated edge cases, state machines and APIs, then argued against its own drafts when prompted ("what breaks this?", "what would a CFO reject?")	The logged decisions (next page), the North Star metric, and what's deliberately out of scope. Every assumption is labelled as one, and demand is marked unproven with a kill switch.
Build	Pixel-sampled brand colours from live app assets, wrote tokens and a component library, previewed every component and synced it to Claude Design, then generated the prototype	The non-negotiables: no typeable amount field anywhere, hidden not disabled controls, 56 px touch targets, icon plus vernacular label, amounts in the largest type on screen.
Audit	Walked every screen against every PRD rule and found 39 issues, each with a PRD reference and a fix	Severity and sequencing. Which issues are PRD non-compliance (blockers) and which are taste, plus the gate each one has to clear.

My operating rules for working with AI

1

Give it a role and a sceptic's job, not a writing job. The project brief told AI to pressure-test every assumption with me.

2

No claim without a source or an "assumption" label. The PRD opens with 12 explicit assumptions; the research doc cites every benchmark.

3

AI drafts, I decide, and the decision is logged with the alternative I rejected and its cost.

4

AI checks AI. A fresh pass audits the build against the spec. It caught what the builder pass missed.

5

Chain the artifacts. Research → PRD → design system → prototype → UAT. Each is written so the next stage (human or AI) can use it directly.

✓

Delivery complete

✓

2 items delivered

🛒

3 items in return pouch

📷

₹794 collected (cash)

₹68 earned

+ ₹34 when the pouch is scanned at the store

Partial-delivery success screen. It shows the pending return fee *and* its release condition, a lesson from rider distrust of opaque pay at Zepto.

03 Where judgment mattered: decisions I pressure-tested with AI

AI is good at generating options and attacking them. The call, and owning its trade-off, stays with the PM.

QUESTION	OPTIONS ON THE TABLE	MY CALL	WHY, AND WHAT IT COSTS
Who calculates the new bill?	Query each brand's ERP / promo engine at the door, or a Zippee pricing engine using order-time data	ZIPPEE ENGINE	150 brands would mean 150 ERP integrations, and the doorstep would inherit their latency and downtime. One deterministic engine responds in milliseconds and covers all brands from day one. Cost: minimum-order coupon rules can't be re-checked after returns; this is disclosed to brands at opt-in.
What proves success?	"Value saved on rejections" vs. revenue realisation rate against matched comparison stores	REALISATION RATE	"Value saved" grows when rejections grow, which is Goodhart's law in action. Realisation is self-guarding: saved sales push it up and induced rejections pull it down. It's also what Zippee sells to brands (fewer RTOs, expressed in money).
How is the rider paid for the return leg?	Flat fee per returned unit vs. % of the order's forward payout	50% OF FORWARD	A flat per-unit fee inverts on cheap items and rewards returns. The fee is released only when the store verifies the returns, so there is never a clawback on unverified goods.
Can money move backwards at the door?	Allow refunds and adjustments, or never	NEVER	No refunds or bill edits at the door, payment only after OTP, and the server is the only calculator. Prepaid refunds always go through the brand. Normal change for cash is unaffected.
What ships first?	Everything at once, or split money (M1) from goods (M2)	M1 FIRST	M1 is pure software where the daily loss is. M2 is a process change across ~125 stores. Demand is unproven, so Phase 0 measures it: under 2% of POD orders means we stop after M1.
Approval calls to ops?	Keep ops sign-off per partial, or rider self-serve with controls	SELF-SERVE	The feature exists to kill the ops call. OTP, photos, geofence tiers, audit thresholds and trend-based auto-pause replace human sign-off.

How success is measured

North Star: revenue realisation rate (delivered and paid merchandise ÷ dispatched), pilot stores vs. matched comparison stores.

- **Adoption:** ≥95% of doorstep rejections resolved in-app with no ops call
- **Money integrity:** <0.5% of partial orders fail to reconcile
- **Stock integrity:** ≥99% of returned units scanned back within 24 h
- **Early warning:** item-rejection rate ≤1.25× comparison stores; auto-pause above 1.5× for 2 weeks

Phased launch with kill criteria

- **Phase 0 (2 wks):** ops tags every partial call; a 12-rider usability test. Gate: addressable volume ≥2% of POD orders, or shelf.
- **Phase 1 pilot, M1 (4 wks):** 3 stores + 3 comparison stores, 5 brands. Gate: realisation beats comparison with statistical confidence.
- **Phase 2, M2 + scale:** return scanning and dock QC network-wide.
- **Phase 3:** all brands by opt-in addendum.

04 AI as a pre-development QA team: what the audit caught

The prototype looked finished. A separate AI pass, told to test it only against the PRD, found 39 issues. These are the five blockers, plus one design-system trap, that would otherwise have reached development.

SEV.	WHAT THE AUDIT FOUND	WHY IT MATTERS
B1	An expired UPI QR unlocked the cash button instantly, skipping the "PSP confirms not paid" check	A customer who pays at second 299 with a slow webhook gets charged twice. Fix: run the same PSP check the void flow already uses.
B4	OTP "attempts left" reached 0, but the field kept accepting guesses	Consent could be brute-forced. Fix: lock after 5 attempts, force the voice OTP or signature path, and log it.
B5	The 2-minute payment-verification timer hit 0:00 with no end state	The rider is stranded at the door. Fix: a decision sheet with two exits: void the QR and collect cash, or convert to full RTO.
B2-B3	Perishable lock and bundle auto-return existed in components but were never wired into the flow	These are the most brand-risky rules, and they were invisible. Fixing them was wiring, not design work.
H4	The design system's default return reasons included "Too expensive"	That default invites price negotiation at the door. Fix: correct it in the design system itself, so no future screen inherits it.

Payment safety designed in from the start

- **Amount-locked dynamic UPI QR**, tagged with order + revision ID; one live QR at a time, maximum 3 regenerations
- **Cash disabled while a QR is live**; a late UPI after cash is auto-detected and refunded
- **Idempotency key** (order + revision + step) on every money call; PSP callbacks deduplicated
- **Frozen calculation per revision** (inputs, rule version, outputs) as the dispute record
- **Collect before handover**; daily PSP ↔ ledger reconciliation, and anything unmatched becomes a ticket within 48 h

Impact, stated honestly

This was an assignment, so there's no production data. What I can measure is the process:

- One PM produced a research-backed, dev-ready package that normally needs a PM, a researcher, a designer and a QA pass
- Coded component library scaffolded, previewed and synced to Claude Design in **under an hour** (per git history)
- **5 blockers + 7 high-severity issues** were found before any engineering time was spent
- Outcome metrics, guardrails and kill criteria were defined **before** the build, not after

What I've learned about leveraging AI as a PM

- **The leverage is in the chain, not the prompt.** A research doc written so a PRD can cite it, and a PRD written so a prototype and an audit can be checked against it, compounds far more than any single clever prompt.
- **AI is confident about happy paths and blind to its own gaps** unless a second pass is asked to audit against a spec. Treat it like a team: builder and reviewer are different roles.
- **My time moved from producing documents to making calls.** Pricing ownership, metric design, rider pay, sequencing and kill criteria are where the PM adds value.

Why this is relevant to Razorpay

The hardest part of this feature turned out to be a payments problem: collecting an amount that changed seconds earlier, over UPI, on a weak network, without double collection, and reconciling it cleanly for 150+ merchants. That's the kind of problem I want to keep working on, and the way I'd work on it is the one shown here: **AI for breadth and speed, a PM for judgment and accountability.**